

Git Internals – How Git Really Works

1. Unix File System – The Foundation

In Unix/bash, a forward slash `/` is always a directory separator.

```
# this requires directory "left" to already exist  
echo "content" > left/right.txt
```

```
# create directory first, then file  
mkdir -p left  
echo "content" > left/right.txt
```

```
"left/right.txt" → directory "left" + file "right.txt"  
"left/right"    → directory "left" + file "right" (no extension)
```

This is a relative path.

The slash ALWAYS means directory separator.

This is the key to understanding Git branches.

2. Git Uses the File System as a Database

Every branch in Git is just a **text file containing one line** – a commit hash.

```
# see what a branch actually is
cat .git/refs/heads/main

# output:
a3f2c11b9e4f2c3d8e7f6a5b4c3d2e1f0a9b8c7d
```

That's it. One hash. That's a branch.

3. Structure of .git/refs

```
tree .git/refs
```

```
.git/refs/
├── heads/           ← ALL local branches
│   ├── main
│   ├── feature/
│   │   ├── tiles
│   │   └── stamina
│   └── fix/
│       └── physics
├── remotes/        ← remote branches
│   └── origin/
│       ├── HEAD
│       └── main
└── tags/           ← tags
    └── v1.0
```

heads/

4. Where is HEAD?

```
# HEAD is a separate file, not in refs/  
cat .git/HEAD  
  
# output when on main:  
ref: refs/heads/main  
  
# output in detached HEAD state:  
a3f2c11b9e4f2c3d8e7f6a5b4c3d2e1f0a9b8c7d
```

HEAD always tells you **where you are right now**.

5. Where is "origin"?

```
# origin is NOT in .git/refs  
# origin is in .git/config
```

```
cat .git/config
```

```
[remote "origin"]  
  url = https://github.com/username/repo.git  
  fetch = +refs/heads/*:refs/remotes/origin/*
```

```
[branch "main"]  
  remote = origin  
  merge = refs/heads/main
```

```
origin    =  a name (alias) for a URL  
              stored in .git/config  
              NOT in .git/refs
```

You could name it anything:

6. Branch Naming Convention – Why Slashes Work

Because Git uses the file system as a database,
a slash in a branch name creates a **real subdirectory**.

```
git checkout -b feature/tiles # or switch
```

```
# Git physically creates:  
# .git/refs/heads/feature/tiles  
#           └──┬──┘  
#           directory file
```

```
# verify it yourself  
tree .git/refs/heads
```

```
# output:  
heads/  
├── main  
└── feature/  
    └── tiles
```

7. Full Reference Notation

Every branch has a full address called a **ref**:

<code>refs/heads/main</code>	← local branch main
<code>refs/heads/feature/tiles</code>	← local branch feature/tiles
<code>refs/remotes/origin/main</code>	← remote branch main on origin
<code>refs/remotes/origin/feature/tiles</code>	← remote branch on origin
<code>refs/tags/v1.0</code>	← tag

Git lets you use shortcuts:

```
# these are identical:  
git merge refs/remotes/origin/main  
git merge origin/main  
  
# these are identical:  
git checkout refs/heads/feature/tiles  
git checkout feature/tiles
```

8. See Everything With One Command

```
# show all refs with their hashes
git show-ref

# output:
a3f2c11 refs/heads/main
b7d4e22 refs/heads/feature/tiles
c9e1f33 refs/remotes/origin/main
d4e5f66 refs/remotes/origin/feature/tiles
e5f7g88 refs/tags/v1.0
```

9. Summary – The Mental Model

Branch = a file in `.git/refs/heads/`
containing one commit hash

Remote = a name + URL in `.git/config`
its branches live in `.git/refs/remotes/`

HEAD = `.git/HEAD`
points to your current branch

origin/main = shortcut for `refs/remotes/origin/main`
two parts separated by slash:
origin = remote name
main = branch name on that remote

feature/tiles = ONE branch name containing a slash
Git creates a real subdirectory
slash here is NOT a remote separator
it is part of the branch name

10. Commands Cheat Sheet

```
# see all local branches  
git branch
```

```
# see all remote branches  
git branch -r
```

```
# see everything  
git branch -a
```

```
# see all refs with hashes  
git show-ref
```

```
# see physical structure on disk  
tree .git/refs
```

```
# see what a branch actually is  
cat .git/refs/heads/main
```

```
# see where you are  
cat .git/HEAD
```

```
# see remote URLs and tracking  
cat .git/config
```

```
# see physical directory structure (bash)  
find .git/refs -type f
```

NEXT

Git Commands – Understanding the Details

1. git checkout Was Doing Too Much

Originally, `git checkout` handled everything:

```
git checkout feature/tiles      # switch branch
git checkout -b feature/tiles   # create and switch branch
git checkout -- main.c          # restore a file
git checkout refs/heads/feature/tiles # full ref notation
```

One command, too many responsibilities.

This caused confusion, especially for beginners.

2. Git 2.23 Split checkout Into Two Commands

```
git checkout (old)
├── git switch    ← switching and creating branches
└── git restore  ← restoring files
```

git switch – branch operations only

```
# switch to existing branch
git switch feature/tiles
```

```
# create and switch
git switch -c feature/tiles
```

```
# what git switch CANNOT do:
```

```
git switch refs/heads/feature/tiles    ✗ does not work
git switch -- main.c                   ✗ does not work
```

3. The Slash – Two Different Meanings

This is one of the most confusing things in Git.

The slash `/` means something **different** depending on context.

Context 1 – Local ref pointer (ONE argument)

```
git merge origin/main
#
#   _____
#   ONE token
#   local pointer to a remote branch
#   exists as a file at:
#   .git/refs/remotes/origin/main
#
#   origin = remote name (part of the pointer name)
#   main    = branch name (part of the pointer name)
#   the slash is PART of the name, not a separator
```

Context 2 – git fetch arguments (TWO arguments)

4. git fetch – Full Syntax

```
git fetch <remote> <branch>
```

```
# fetch ALL branches from origin  
git fetch origin
```

```
# fetch only main from origin  
git fetch origin main
```

```
# fetch only feature/tiles from origin  
git fetch origin feature/tiles
```

```
#  
#           _____  
#           this is the branch name on the remote  
#           the branch name contains a slash  
#           but it is still ONE argument (arg2)
```

```
# fetch from a different remote  
git fetch upstream main  
git fetch superman feature/tiles
```

5. Default Parameters – How Git Fills Them In

When you omit arguments, Git reads `.git/config` to figure out what you meant.

```
# full explicit command:
git fetch origin
git merge origin/main

# shortcut when branch tracking is configured:
git pull          # git knows: fetch origin, merge origin/main
git push          # git knows: push to origin/main

# Git checks .git/config:
cat .git/config
```

```
[remote "origin"]
    url = https://github.com/username/repo.git
```

```
[branch "main"]
```

6. Summary – The Slash Rules

CONTEXT	SLASH MEANING	EXAMPLE
local ref pointer	part of pointer name	origin/main
branch name with category	part of branch name	feature/tiles
git fetch / push / pull	NO slash	git fetch origin main
full ref notation	directory separator	refs/heads/feature/tiles

```
# these mean the same thing:  
git merge origin/main  
git merge refs/remotes/origin/main
```

```
# these are NOT the same syntax:  
git merge origin/main      ← ONE argument (local pointer)  
git fetch origin main      ← TWO arguments (remote + branch)
```

#####

merge

#####

git fetch and git merge – How They Actually Work

The Key Idea

```
fetch    = update your map of what exists on GitHub
merge    = apply those changes to your local branch
pull     = fetch + merge in one command
```

What fetch Actually Does

`git fetch` updates **one specific location** inside `.git/refs/`:

```
git fetch origin
```

UPDATES:

<code>.git/refs/remotes/origin/main</code>	✓ updated to latest hash
<code>.git/refs/remotes/origin/feature/tiles</code>	✓ updated to latest hash

DOES NOT TOUCH:

<code>.git/refs/heads/main</code>	✗ unchanged
your working files	✗ unchanged

Your local branch is untouched.

You have a fresh map of what is on GitHub,
but nothing in your project has changed yet.

fetch accepts two parameters

What merge Does

`git merge origin/main` takes the hash stored in `.git/refs/remotes/origin/main` and applies it to your local branch.

BEFORE merge:

```
.git/refs/  
├── heads/  
│   └── main          ← old hash (your local state)  
└── remotes/  
    └── origin/  
        └── main      ← new hash (what is on GitHub)
```

AFTER `git merge origin/main`:

```
.git/refs/  
├── heads/  
│   └── main          ← updated to new hash ✓  
└── remotes/  
    └── origin/  
        └── main
```

Three Possible Outcomes of merge

Outcome 1 – Fast-forward (nothing to merge)

This happens when you have not added any new commits locally since the last time you were in sync with GitHub.

GitHub has moved forward, you have not.

BEFORE:

local main: A → B → C
 ↑
 HEAD

origin/main: A → B → C → D → E
 ↑
 origin/main

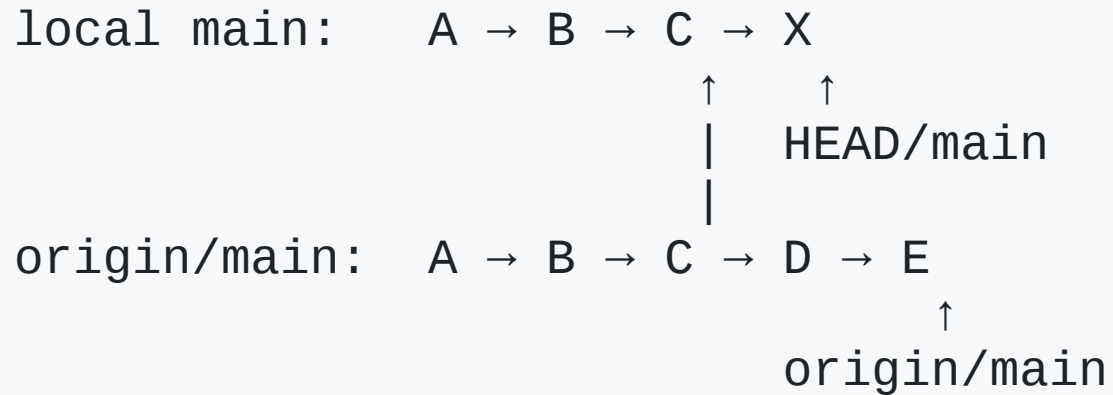
```
git fetch origin
git merge origin/main
```

Outcome 2 – Merge commit (both sides have new commits)

This happens when you and someone else both added commits since the last common point.

Different files were edited, so Git can merge automatically.

BEFORE:



C is the last common commit (common ancestor)

X is your commit

D and E are commits from GitHub

git fetch origin

Outcome 3 – Conflict (same file edited in both places)

This happens when you and someone else edited the **same lines in the same file**.

Git cannot decide which version to keep.

BEFORE:

```
local main:  A → B → C → X    ← X edits line 42 of main.c
                ↑      ↑
                |      HEAD/main
                |
origin/main: A → B → C → D → E ← E edits line 42 of main.c
                ↑
                origin/main
```

C is the last common commit
both X and E touch the same line

```
git fetch origin
git merge origin/main
```

git pull = git fetch + git merge

```
# these two blocks are identical:
```

```
# block 1 - explicit:
```

```
git fetch origin
```

```
git merge origin/main
```

```
# block 2 - shortcut:
```

```
git pull origin main
```

```
# when tracking is configured (after git push -u origin main):
```

```
git pull
```

```
# Git reads .git/config and expands to:
```

```
# git fetch origin
```

```
# git merge origin/main
```

Summary

COMMAND	UPDATES	TOUCHES YOUR FILES?
git fetch origin	refs/remotes/origin/*	NO
git fetch origin main	refs/remotes/origin/main	NO
git merge origin/main	refs/heads/main	YES
git pull origin main	both of the above	YES

fetch	→	updates .git/refs/remotes/	(your map of GitHub)
merge	→	updates .git/refs/heads/	(your local branch)
pull	→	does both in one step	

Outcome comparison

OUTCOME	TRIGGER	RESULT
Fast-forward	only GitHub has new commits	HEAD moves forward no new commit created